

PHYLOPOMP.JL Development Branch

Summary of Changes Merged from `devel` into `atpabuser-devel`

Auto-generated Documentation

July 17, 2026

Contents

1	Overview	2
2	New Feature: Soft-Guided SEIR Filter (SoftSEIR)	2
2.1	Design Philosophy	3
2.2	Key Implementation Details	3
2.3	Documentation	3
3	Filter-Guide Infrastructure Refactoring	3
3.1	Type Parameter Changes	3
3.2	New Allocation-Free Relative Hazard API	4
3.3	Removed: <code>demekron</code> (Non-Mutating)	4
3.4	Removed: <code>ell(::BitSet)</code>	4
3.5	New Exports	4
4	Hard-Guided SEIR Filter Refactoring (HardSEIR)	4
5	Shared SEIR Utilities (<code>seir_funs.jl</code>)	5
5.1	Sampling Logic Fix	5
5.2	Time-Aware <code>rprocess</code>	5
6	Guided SEIR Filter (<code>seir_guided.jl</code>) — Inlining	6
7	MERS Naive Filter Bug Fixes	6
8	Finite-State Markov Process (<code>fsmarkov.jl</code>)	6

9	Genealogy and Newick Formatting	6
9.1	Keyword Argument Simplification	6
9.2	Removed Method	7
10	Dependency and Build Changes	7
10.1	POMP.jl Dependency	7
10.2	Version Bump	7
10.3	Makefile	7
11	CI/CD Workflow Changes	7
11.1	CI Matrix Restructuring (CI.yml)	7
11.2	Documentation Workflow (Documentation.yml)	8
12	Test Suite Updates	8
13	Summary of Files Changed	8

1 Overview

This document summarizes the changes introduced by merging the upstream `devel` branch (from `kingaa/PhyloPOMP.jl`) into the local `atpabuser-devel` branch. The merge encompassed 16 commits affecting 25 files, with 647 lines added and 232 lines removed. All changes applied as a clean fast-forward merge.

The changes fall into five major categories:

1. A new “soft-guided” SEIR filter module (`SoftSEIR`).
2. Significant refactoring of the filter-guide infrastructure for performance and generality.
3. Bug fixes in the MERS naive filter and SEIR sampling logic.
4. Dependency and CI/CD updates.
5. Code simplification and cleanup throughout.

2 New Feature: Soft-Guided SEIR Filter (`SoftSEIR`)

A new module `SoftSEIR` has been added in `src/examples/seir_soft.jl` (88 lines), providing an alternative guided particle filter for the SEIR model.

2.1 Design Philosophy

The soft-guided filter uses a filter guide to steer population-process events preferentially onto or away from particular lineage branches, but crucially, *the overall event rate remains equal to that of the underlying population process*. This contrasts with the “hard-guided” filter (HardSEIR), where the guide modifies the actual event rates.

2.2 Key Implementation Details

- Uses the new `relhaz_alloc / relhaz! / choose_branch` interface (see Section 3) for efficient, allocation-free guide queries.
- The `regular_part!` function simulates the Gillespie dynamics between genealogical events, applying guide-based branch selection for transmission and progression events involving tracked lineages.
- Six event channels are handled: transmission (off-lineage and on-lineage), progression (off-lineage and on-lineage), recovery, and waning immunity.
- Includes its own test suite (`test/seir_soft.jl`) with simulation, particle filter, and benchmark tests.

2.3 Documentation

A new section has been added to `docs/src/seir.md` documenting SoftSEIR with an example that constructs a `pomp` object and runs a 1000-particle filter.

3 Filter-Guide Infrastructure Refactoring

The guide system (`src/guide.jl`, `src/fsmarkov.jl`) has undergone substantial refactoring for both performance and type-level generality.

3.1 Type Parameter Changes

All core structs now carry an additional type parameter `N` representing the number of demes, which is resolved at compile time rather than being computed at runtime via `length(instances(D))`.

Struct	Before	After
<code>FMarkovProc</code>	<code>FMarkovProc{F,D}</code>	<code>FMarkovProc{F,N,D}</code>
<code>GuideNode</code>	<code>GuideNode{F,D}</code>	<code>GuideNode{F,N,D}</code>
<code>Guide</code>	<code>Guide{F,D}</code>	<code>Guide{F,N,D}</code>

The `N` parameter enables the compiler to specialize matrix operations on the known deme dimension, replacing runtime `length(instances(D))` calls with compile-time constants.

3.2 New Allocation-Free Relative Hazard API

Three new functions and one helper have been added to enable pre-allocated, in-place computation of relative hazards:

`relhaz_alloc(g, n)` — Allocates a dictionary of vectors (keyed by deme-transition pairs (i, j)) sized for guide node `n`. Called once per Gillespie interval.

`relhaz!(r, t, g, n)` — Fills the pre-allocated dictionary `r` in-place with the relative hazards at time `t`. Replaces repeated allocation in the inner Gillespie loop.

`sum_relhaz(r, n, cols, i, j)` — Returns the sum of relative hazards for the $i \rightarrow j$ transition across lineages in coloring `cols`.

`choose_branch(r, n, cols, i, j)` — Selects a branch for a deme-switching event using the pre-computed hazards, returning both the chosen branch and its selection probability.

These replace the previous pattern where `relhaz` was called repeatedly inside the Gillespie loop, each time allocating new vectors.

3.3 Removed: demekron (Non-Mutating)

The non-mutating `demekron(F, d)` and `demekron(d)` functions have been removed. Only the in-place `demekron!(v, d)` remains, reducing unnecessary allocations.

3.4 Removed: ell(::BitSet)

The method `ell(y::BitSet)` (which was simply `length(y)`) has been removed from `src/coloring.jl`.

3.5 New Exports

The module now exports the new functions:

```
export Guide, guide, relhaz, relhaz_alloc, relhaz!,
       sum_relhaz, demekron!, choose_branch
```

4 Hard-Guided SEIR Filter Refactoring (HardSEIR)

The hard-guided filter in `src/examples/seir_hard.jl` has been substantially simplified (145 lines reduced to approximately 95 lines of core logic):

- The local `event_rates!` function has been removed. The module now uses the shared `event_rates!` from `seir_funs.jl`, which decomposes rates into modular per-event-type functions.
- The `regular_part!` function now accepts explicit `t` and `tf` time arguments instead of extracting them from the guide node, enabling the caller to control the integration window.

- Guide queries use the new pre-allocated `relhaz!` / `choose_branch` API instead of per-call allocation.
- Unnecessary `@assert false` branches after exhaustive conditionals have been removed.

5 Shared SEIR Utilities (`seir_funs.jl`)

The shared utility file `src/examples/seir_funs.jl` has been significantly expanded (from ~ 20 to ~ 100 lines of new content) with modular event-rate functions:

`transmission!(alpha, pi; ...)` — Computes rates and pre-boost probabilities for transmission events, split into off-lineage and on-lineage channels.

`progression!(alpha, pi; ...)` — Analogous function for $E \rightarrow I$ progression events.

`recovery!(alpha, pi; ...)` — Recovery ($I \rightarrow R$) rate computation.

`waning!(alpha, pi; ...)` — Waning immunity ($R \rightarrow S$) rate computation.

`sampling(; ...)` — Returns the total sampling rate $(\psi + \chi)I$.

`event_rates!(alpha, pi; ...)` — Master dispatcher that calls all of the above and accumulates the total “decay” (the difference between true and boosted rates).

5.1 Sampling Logic Fix

The sampling logic at genealogical sample nodes has been corrected in both `seir_funs.jl` and `seir_naive.jl`:

- Previously, the code computed $\text{ll} += \log((\psi + \chi) \cdot I)$ before choosing the sampling type, then adjusted the log-likelihood with a conditional on the number of tracked lineages.
- Now, the sampling type is chosen first via `rcateg([ $\psi$ ,  $\chi$ ], true)`, the selection probability is subtracted ($\text{ll} -= \log(p)$), and the log-likelihood contributions are:
 - Non-destructive: $\log(\psi \cdot (I - \ell_I))$
 - Destructive: $\log(\chi \cdot I)$

5.2 Time-Aware rprocess

The `rprocess` callback in `filter_pomp` now receives `t` and `dt` keyword arguments, computing $t_f = t + dt$. The `regular_part!` call is skipped when $t \geq t_f$, avoiding zero-length Gillespie intervals.

6 Guided SEIR Filter (`seir_guided.jl`) — Inlining

The `GuidedSEIR` module (`src/examples/seir_guided.jl`) has been expanded from a thin wrapper around `seir_funs.jl` to a self-contained module (~170 lines). The key functions — `knowledge!`, `singular_part!`, and `filter_pomp` — are now defined directly in the module rather than included from `seir_funs.jl`. This allows the guided variant to diverge from the shared utilities where needed (e.g., its `singular_part!` uses a slightly different sampling branch structure that retains the older two-branch `rcateg` pattern).

7 MERS Naive Filter Bug Fixes

Several correctness fixes have been applied to `src/examples/mers_naive.jl`:

1. **Cross-species transmission rates:** The factor of $1/2$ was incorrectly applied to the total cross-species rate (e.g., `Beta_hc * Sh * Ic / Nc / 2.0`). It is now correctly applied only to the individual branch weights passed to `rcateg` (e.g., `0.5*lambda_hc`), while the full rate is used in the log-likelihood.
2. **Branch selection probability:** The `rcateg` calls now use the `true` flag to return the selection probability, which is properly subtracted from the log-likelihood (`ll -= log(p)`). Previously, the returned rate value was used directly, conflating the rate with the selection probability.
3. **Missing log-likelihood terms:** Two `ll += log(ellx)` terms were added for cross-species events in the regular part (events $k = 4$ and $k = 6$), accounting for the probability of selecting a tracked lineage.

8 Finite-State Markov Process (`fsmarkov.jl`)

- The `FMarkovProc` struct gains the `N` type parameter (number of demes), computed in the constructor as `N = length(instances(D))`.
- The `forward_action` and `relhaz_action` methods now use `@assert size(X,1) == M` (where `M` is the type parameter) instead of a runtime `if/error` check. This provides clearer error messages and enables the compiler to elide the check when the size is statically known.

9 Genealogy and Newick Formatting

9.1 Keyword Argument Simplification

Throughout `src/genealogy.jl` and `src/newick.jl`, all calls of the form `round(x, sigdigits=sigdigits)` have been simplified to `round(x; sigdigits)` using Julia’s shorthand keyword syntax. This is a purely stylistic change with no behavioral impact.

9.2 Removed Method

The standalone `Base.show(g; kwargs...)` method (which dispatched to `show(stdout, g; kwargs...)`) has been removed from `genealogy.jl`. The two-argument `Base.show(io, g; kwargs...)` method remains.

10 Dependency and Build Changes

10.1 POMP.jl Dependency

The `Project.toml` now pins `PartiallyObservedMarkovProcesses` to the upstream `devel` branch via a `[sources]` entry pointing to `https://github.com/kingaa/PartiallyObservedMarkovProcesses.jl` with `rev = "devel"`.

The compatibility specifier has been updated from `"0.6.3"` to `"0.7"`.

The same source entry has been added to `test/Project.toml`.

10.2 Version Bump

The package version has been bumped from `0.0.11-dev` to `0.0.12-dev`.

10.3 Makefile

The test precompilation step now includes `--check-bounds=yes` for stricter bounds checking during test builds.

11 CI/CD Workflow Changes

11.1 CI Matrix Restructuring (CI.yml)

The GitHub Actions CI matrix has been restructured:

- The nested `matrix.config.{os, version, arch, R}` structure has been flattened to top-level matrix keys.
- A new `threads` dimension has been added with values `"1"` and `"2"`, enabling multi-threaded test runs. Julia 1.12 is excluded from the single-thread configuration (only runs with 2 threads).
- The `JULIA_NUM_THREADS` environment variable is set from the matrix for the test step.
- Job names now include the thread count for clarity.

11.2 Documentation Workflow (Documentation.yml)

The documentation workflow now triggers on pull requests in addition to pushes to `master` and tags.

12 Test Suite Updates

- **New:** `test/seir_soft.jl` — full test suite for the `SoftSEIR` module including simulation, particle filtering, and benchmarking (49 lines).
- **Updated:** `test/runtests.jl` now includes `seir_soft.jl`.
- **Updated:** Variable renaming in `test/guide.jl` ($hn \rightarrow rn$) for clarity.
- **Updated:** Benchmark loops in `test/seir_naive.jl`, `test/seir_hard.jl`, and `test/mers_naive.jl` are now wrapped in `@time` for wall-clock reporting.

13 Summary of Files Changed

File	Δ Lines	Nature of Change
<code>src/examples/seir_soft.jl</code>	+88	New module
<code>src/examples/seir_guided.jl</code>	+164	Inlined from shared funs
<code>src/examples/seir_funs.jl</code>	+86	Modular event rates
<code>src/examples/seir_hard.jl</code>	-50	Refactored to new API
<code>src/examples/seir_naive.jl</code>	-3	Sampling fix
<code>src/examples/mers_naive.jl</code>	+8	Bug fixes
<code>src/guide.jl</code>	+42	New API, type params
<code>src/fsmarkov.jl</code>	-6	Type params, asserts
<code>src/genealogy.jl</code>	-6	Cleanup
<code>src/newick.jl</code>	± 2	Keyword syntax
<code>src/PhyloPOMP.jl</code>	+2	New exports
<code>src/coloring.jl</code>	-2	Removed <code>ell(::BitSet)</code>
<code>Project.toml</code>	+5	POMP source, version
<code>docs/src/seir.md</code>	+24	SoftSEIR docs
<code>test/seir_soft.jl</code>	+49	New test suite
<code>test/runtests.jl</code>	+1	Include soft test
<code>.github/workflows/CI.yml</code>	+13	Thread matrix
<code>.github/workflows/Doc.yml</code>	+1	PR trigger
<code>Makefile</code>	± 1	Bounds checking